



Mentalidad: de script a sistema mantenible

Esta unidad **no va solo de Python**. Va de empezar a pensar como alguien que construye **sistemas de IA mantenibles**.

El objetivo pedagógico real es que el alumno deje de escribir:

- scripts gigantes
- lógica mezclada
- prompts hardcodeados
- código frágil

y empiece a:

- separar responsabilidades
- validar datos
- controlar el flujo
- gestionar estado
- estructurar proyectos

NO vs Sí (la idea clave)

NO

- `todo_en_main.py`
- "si funciona, no lo toco"
- prompts pegados en medio del código
- funciones con efectos por todas partes

SÍ

- `config` (constantes y límites)
- `prompts` (plantillas de texto)
- `logic` / `services` (funciones que procesan)
- `state` (historial, sesión, contadores)
- `main` (orquestración mínima)

Qué significa “separar responsabilidades”

Un sistema bien separado te permite cambiar una parte sin romperlo todo:

- cambiar límites de validación sin tocar lógica
- cambiar un prompt sin buscarlo por todo el proyecto
- cambiar “cómo guardo el historial” sin reescribir validaciones

Puente con IA generativa (sin usar API aquí)

Más adelante (Prompt & Context Engineering):

- el **usuario** puede mandar datos sucios → hay que validar
- el **modelo** también puede devolver texto inesperado → hay que validar/parsing/ramificar
- el **historial** en memoria se convierte en “contexto”

En esta unidad aprenderás el esqueleto: **validación + flujo + módulos + estado**.